

Torch2Gemmini

End-to-End Deployment of PyTorch Models
on FPGA-Based Gemmini Accelerator

Noel Manuel | EP22B035

Undergraduate Research (UGRC) · IIT Madras

RISC-V

MLIR

Gemmini

PyTorch

FPGA

Problem Statement & Objectives

THE CHALLENGE

- Most Machine learning models are written in Pytorch
- Existing Gemmini programs are written manually in C/C++
- No end-to-end automated pipeline from Python → FPGA

OBJECTIVES

- Build complete PyTorch → Gemmini pipeline
- Add torch.int_mm support in Torch-MLIR
- Fix int8 tensor lowering in Buddy-MLIR
- Enable VC707 FPGA deployment via UART-TSI
- Validate with matrix mul, MLP & LeNet

System Architecture

CHIPYARD ECOSYSTEM

Rocket Processor

RISC-V in-order CPU, controls Gemmini via RoCC interface

Gemmini Accelerator

16×16 systolic array · int8 mul · int32 accumulate

VC707 FPGA

Deployment target · 25 MHz · UART-TSI loading flow

COMPILER STACK

Torch-MLIR

Converts PyTorch to linalg MLIR dialect

Buddy-MLIR

Lowers linalg → Gemmini dialect → LLVM dialect

LLVM / CHIPYARD GCC

Generates RISC-V machine code & ELF binary

Compilation Flow



Key Modifications:

Added torch.int_mm in Torch-MLIR · Fixed int32→int8 buffer recovery in Buddy-MLIR · Enabled UART-TSI on VC707

Experimental Results

VC707 FPGA · Rocket + Gemmini · 16×16 systolic array · 25 MHz · int8 mul / int32 acc

Matrix Mul
8×8

8.25×

speedup

CPU: 38,593 cycles

Gemmini: 4,678 cycles

MLP Inference
MNIST (784→128→10)

637.8×

speedup

CPU: 27,172,259 cycles

Gemmini: 42,600 cycles

LeNet Inference
MNIST CNN

43.87×

speedup

CPU: 73,482,069 cycles

Gemmini: 1,674,705 cycles

What did I learn from the project

I understood that research systems are incomplete and evolving. A lot of the work involved reading code-bases, identifying missing operator support, and extending frameworks.

Getting a workflow to run in simulation is very different from deploying it on real FPGA hardware.

A bug that you worked days for , might have a 2 line fix.

Conclusion

- Built an end-to-end framework: PyTorch → Torch-MLIR → Buddy-MLIR → RISC-V ELF
- Contributed: torch.int_mm support, int8 buffer recovery pass, VC707 UART-TSI flow
- Demonstrated 638× MLP speedup and 44× LeNet speedup on real FPGA hardware
- Proves viability of open-source compiler + accelerator stack for ML deployment

Future Work

Expand operator coverage · Automate wrapper generation · Support larger models · Generalise to arbitrary PyTorch programs